Exam Notes: Model Types

June 6, 2026

Contents

1 Synchronous Models **4**

1.1 Core assumption 4

1.2 Typical properties 4

1.3 Why it matters 4

1.4 Exam prompts 4

2 Synchronous Model **5**

2.1 Notation cheat-sheet 5

2.2 (a) Fill in an execution trace 5

2.3 (b) Find inputs reaching a target state 6

2.4 (c) Redraw as an extended state machine 6

2.5 Quick facts for short-answer questions 7

3 Synchronous Model — Part 2 **8**

3.1 Combinational components & await-dependencies 8

3.2 Composition operations (block-diagram exercise) 8

3.3 Task graph, precedence & schedule 9

3.4 From component to transition system (S, Init, Trans) 9

3.5 Invariants vs. inductive invariants (key exercise) 9

4 Asynchronous Models **11**

4.1 Core assumption 11

4.2 Typical properties 11

4.3 Why it matters 11

4.4 Exam prompts 11

5 Asynchronous Model **12**

5.1 What you are given (anatomy of a process) . . . 12
5.2 Parallel composition (the main exam question) . . . 12
5.3 Fully worked template (Plus/Minus) . . . 13
5.4 Fairness questions (short-answer) . . . 13
5.5 Mutual exclusion / Peterson facts (short-answer) . . . 14
5.6 Reachability / `Post` (verifying a property) . . . 14

6 Safety Requirements **15**
6.1 Core definitions to quote verbatim . . . 15
6.2 Three logical facts that answer most sub-questions . . . 15
6.3 The main exam question: Reachable States (a)–(e) . . . 16
6.4 Fully worked template . . . 16
6.5 Symbolic representation (transition formula) . . . 17
6.6 Checking an invariant by reachability (`Post` / BFS) . . . 17

7 Safety Requirements — Part 2 **18**
7.1 Safety monitor: setup to quote . . . 18
7.2 The main exam question: Safety Monitor (a)–(b) . . . 18
7.3 Fully worked template . . . 19
7.4 Symbolic verification machinery (short-answer) . . . 19
7.5 Checking an invariant symbolically (`Post` + BFS) . . . 20
7.6 Data structures (one-liners) . . . 20

8 Timed Models **21**
8.1 Core assumption . . . 21
8.2 Typical properties . . . 21
8.3 Why it matters . . . 21
8.4 Exam prompts . . . 21

9 The Timed Model **22**
9.1 What a timed process is . . . 22
9.2 The two kinds of transition . . . 22
9.3 Tracing a run (core exam skill) . . . 23
9.4 Modelling a timing spec (guard + invariant pattern) . . . 23

9.5 Parallel composition of timed processes . . . 24
9.6 From timed process to timed automaton . . . 24
9.7 Fischer's protocol (timing-based mutual exclusion) . . . 24

10 Timed Automata & UPPAAL 25

10.1 Anatomy of a timed automaton . . . 25
10.2 Guards vs. invariants (don't mix these up) . . . 25
10.3 State & the two kinds of transition . . . 26
10.4 The recurring exam exercise (Exploration of Timed Automata) . . . 26
10.5 Zones & difference constraints . . . 27
10.6 Networks, channels & special locations . . . 27
10.7 UPPAAL query language . . . 27
10.8 Fischer's mutual exclusion (timed protocol) . . . 28

11 Continuous Models 29

11.1 Core assumption . . . 29
11.2 Typical properties . . . 29
11.3 Why it matters . . . 29
11.4 Exam prompts . . . 29

12 Dynamical Systems – Part 1 30

12.1 Does a solution exist / is it unique? . . . 30
12.2 Euler method (the main computation) . . . 31
12.3 Runge–Kutta & truncation error . . . 32
12.4 Equilibria . . . 32
12.5 Stability (Lyapunov) . . . 32

Chapter 1

Synchronous Models

1.1 Core assumption

Definition

A synchronous system assumes known bounds on message delays, process speeds, and clock drift. These bounds make timing part of the model rather than a source of uncertainty.

1.2 Typical properties

- Messages arrive within a known maximum time.
- Local computation steps have a known upper bound.
- Protocols can use timeouts as reliable evidence of failure or delay.

1.3 Why it matters

Key Idea

Many classical distributed algorithms are simpler in a synchronous model because they can distinguish between a slow process and a failed one using timing bounds.

1.4 Exam prompts

- State the assumptions that define synchrony.
- Explain how synchrony supports timeout-based reasoning.
- Compare synchrony with asynchronous communication.

Chapter 2

Synchronous Model

Synchronous Model

Exam cheat-sheet: *execution traces, reaching a target state, extended state machines*

Exam Exercise 1 on synchronous components is almost always the same three tasks on a given component: **(a)** fill in an execution trace, **(b)** find inputs reaching a target state, **(c)** redraw the component as an extended state machine.

2.1 Notation cheat-sheet

- **Component** $= (I, O, S, \text{Init}, \text{React})$: input vars, output vars, state vars, initialisation, reaction code run *once per round*.
- **Reaction:** $s \xrightarrow{i/o} t$ means: in old state s, with input i, produce output o and move to new state t.
- **State** is written as the tuple of state variables in declaration order, e.g. $(mode, x)$. Inputs are written as the tuple of input vars; output after the "/".
- **Events:** `event` $\in \{\bot, \top\}$; `event(nat)` $\in \{\bot, 0, 1, 2, \dots\}$. $\bot$ = absent, $\top$ = present.
- `x?` tests "x present". `x!v` means `x:=v` (and `x!` means `x:=present`). **Unassigned output event $\Rightarrow$ absent ($\bot$) by default.**
- **Combinational** = *no* state variables (e.g. `out:=in1-in2`). **Sequential** = has state variables (e.g. Delay).
- **Deterministic** = single initial state *and* for every (s, i) exactly one (o, t). (`choose`/nondeterminism breaks this.)

2.2 (a) Fill in an execution trace

Recipe (one round at a time):

1. Write the current state tuple and plug the given input values in.

2. Run the update code *top to bottom*, line by line, keeping a scratch value for each variable.
3. Read off the **output** (o) and the **new state** (t).
4. Carry t forward as the start state of the next round; repeat.

Key Idea

Trap: if the code first flips the `mode` and *then* uses `mode`, the later lines use the *new* mode. Always update in code order.

Worked (AddOrMultiply, state $(mode, x)$, input $(in, flip)$): code is "`if flip? then flip mode`; then if `add`: `x:=x+in` else `x:=x*in`; `out:=x`".

$$(\text{add}, 0) \xrightarrow{(2,\perp)/2} (\text{add}, 2) \xrightarrow{(3,\top)/6} (\text{mul}, 6) \xrightarrow{(1,\top)/7} (\text{add}, 7) \xrightarrow{(3,\perp)/10} (\text{add}, 10)$$

Round 2 check: `flip` present $\Rightarrow$ mode becomes `mul`; *then* `x:=x*in`$= 2 \cdot 3 = 6$; `out`$= 6$.

2.3 (b) Find inputs reaching a target state

Recipe:

1. Note the target tuple, e.g. $(\text{mul}, 42)$, and the initial state.
2. Work out the *effect of one input* on each variable (what makes the mode switch, what arithmetic is applied).
3. Find the **minimum length**: if you must reach a mode you cannot start in, you need at least one switching input, so often length ≥ 2.
4. Construct concrete inputs that produce the target value; remember to set each event to $\perp$ or $\top$ as required.

Worked: reach $(\text{mul}, 42)$ from $(\text{add}, 0)$. First input $(a, \perp)$ stays in `add` and sets $x := 0 + a = a$; second input $(b, \top)$ flips to `mul` then sets $x := a \cdot b$. Need $a \cdot b = 42$, e.g.

$$(\text{add}, 0) \xrightarrow{(42,\perp)/42} (\text{add}, 42) \xrightarrow{(1,\top)/42} (\text{mul}, 42).$$

Any factor pair works: $(42, \perp)(1, \top)$, $(21, \perp)(2, \top)$, $(7, \perp)(6, \top)$, ...

2.4 (c) Redraw as an extended state machine

Recipe:

1. The named state variable (e.g. `mode`) becomes the **modes/bubbles**: one bubble per possible value.
2. Every *other* state variable stays a **data variable**, declared with its initial value on the start-arrow (e.g. `int x:=0`). Put the start arrow into the initial mode.

3. For each mode and each relevant case of the inputs, draw a transition. Find its **target mode** by simulating the mode-switch logic, and label it `guard?` $\rightarrow$ `updates; out:=...`. Use a self-loop when the mode does not change.
4. Updates on a switching transition use the behaviour of the *destination* mode (same trap as in (a)).

Worked (AddOrMultiply), modes `add` and `mul`, init `add`, data `int x:=0`:

- `add` $\xrightarrow{\neg\texttt{flip?} \rightarrow x{:=}x{+}in;\ out{:=}x}$ `add` (self-loop)
- `add` $\xrightarrow{\texttt{flip?} \rightarrow x{:=}x{*}in;\ out{:=}x}$ `mul`
- `mul` $\xrightarrow{\neg\texttt{flip?} \rightarrow x{:=}x{*}in;\ out{:=}x}$ `mul` (self-loop)
- `mul` $\xrightarrow{\texttt{flip?} \rightarrow x{:=}x{+}in;\ out{:=}x}$ `add`

(The two switching edges cross: `add`$\rightarrow$`mul` *multiplies*, `mul`$\rightarrow$`add` *adds*, because the arithmetic uses the new mode.)

2.5 Quick facts for short-answer questions

- "Is it combinational?" $\Rightarrow$ Yes iff it has no state variables.
- "Is it deterministic?" $\Rightarrow$ Yes iff one initial state and a unique (o, t) for every (s, i); a `choose` makes it nondeterministic.
- **Synchrony hypothesis:** update code is instantaneous; outputs produced and inputs read in the *same* round; all composed components fire simultaneously each round.
- Event-triggered component reacts only in rounds where its trigger event is present.

Exam triage: *trace?* run code top-to-bottom each round; new mode takes effect immediately. *reach target?* compute mode-switch cost, pick factor pair for numeric target. *ESM?* mode var $\rightarrow$ bubbles; other state vars $\rightarrow$ data on start-arrow; one edge per (mode, guard) case; switching edges use destination-mode arithmetic. *combinational?* no state vars. *deterministic?* unique (o, t) for every (s, i).

Chapter 3

Synchronous Model — Part 2

Synchronous Model — Part 2

Exam cheat-sheet: *composition, await-dependencies, transition systems, invariants*

Part 2 covers **putting components together** (combinational circuits, composition, await-ordering) and **proving safety** (transition systems, invariants, inductive invariants).

3.1 Combinational components & await-dependencies

- **Combinational** = no state variables (e.g. `out:=in1&in2`). Such an output *awaits* the inputs it reads in the same round (written `out awaits in1,in2`).
- Composition is **allowed iff the await-dependency graph is acyclic** (no "combinational cycle"). Feedback like `Relay: out:=in` with `Inverter: in:=~out` is cyclic ⇒ disallowed.

Recipe ("are await-dependencies acyclic?"):

1. For each output, list the inputs it awaits; draw an edge awaited input → output.
2. Add the wiring edges from composition (an output feeds the equally-named input).
3. Look for a cycle. Acyclic ⇒ composition well-defined.

3.2 Composition operations (block-diagram exercise)

Three operations build the product, e.g. $C[out \mapsto temp] \mid D[in \mapsto temp] \setminus temp$:

- **Rename** $C[old \mapsto new]$: rename a variable (used to make two components share a channel name).
- **Hide** $C \setminus c$: make channel c *internal* (local); it no longer crosses the outer box.
- **Parallel** $C \mid D$: a name that is an *output of one* and *input of the other* becomes a connecting channel.

Recipe ("connect / annotate the block diagram"):

1. Apply the renamings to each copy's input/output names first.
2. Matching name (output of one = input of the other) ⇒ draw an internal arrow between them. If that name is hidden ($\backslash c$), keep the arrow *inside* the outer box.
3. Unmatched, non-hidden names ⇒ arrows crossing the outer box (external inputs/outputs of the composition).
4. Annotate every edge with its **type** and (renamed) **name**.

3.3 Task graph, precedence & schedule

- A **task** is an output/update block. Precedence $A' < A$ if A' produces a variable that A awaits.
- Await-compatibility ⇒ the product task graph is acyclic, so at least one schedule exists.
- A **schedule** is a total order of all tasks respecting every edge (a topological sort). If $A' <^{+} A$ then A' comes before A in *every* schedule.

3.4 From component to transition system (S, Init, Trans)

To analyse safety, convert the component:

- State variables stay; the `mode` is an *implicit state variable* (part of the state tuple).
- Input/output variables become **local**; input values are chosen **nondeterministically** each step.
- `Init` = the initial assignment; `Trans` = one round of the reaction for some input choice.

Recipe ("give the reachable part of the transition system"): start at the initial state; repeatedly apply every transition (each mode-switch / input choice) and add any new state; stop when no new states appear. Keep the mode in the tuple.

Worked (ESM: modes A,B; `int x:=0`; A → B does `x:=x+1`; B → A does `x:=x-1`): reachable set is

$$(A, 0) \;\rightleftarrows\; (B, 1).$$

3.5 Invariants vs. inductive invariants (key exercise)

Key Idea

- φ is an **invariant** if *every reachable* state satisfies φ.
- φ is an **inductive invariant** if (1) every *initial* state satisfies φ, and (2) for every transition (s, t), if $s \models \varphi$ then $t \models \varphi$.

- Inductive invariant ⇒ invariant. The converse can fail.

Recipe per sub-question (using the worked ESM above and $\varphi_1 : \ x = 0 \vee x = 1$):

- **Is φ an invariant?** Compute the reachable set, check all satisfy φ. *Yes:* "all reachable states satisfy φ." *No:* give a reachable counterexample state. (Here: yes — both $(A, 0)$ and $(B, 1)$ satisfy φ_1.)
- **Is φ an inductive invariant?** Check base + step. To *disprove*, find *any* state s (need not be reachable) with $s \models \varphi$ but a successor $t \not\models \varphi$. (Here: no — $(A, 1) \models \varphi_1$ but its successor $(B, 2) \not\models \varphi_1$.)
- **Give another invariant φ_2:** anything true on all reachable states, e.g. $x \geq 0$.
- **Give an inductive invariant φ_3:** pin the mode to its value, e.g. $(mode = A \wedge x = 0) \vee (mode = B \wedge x = 1)$.

Recipe to *prove* an inductive invariant:

1. **Base case:** show every initial state satisfies φ.
2. **Inductive step:** take an arbitrary state s with $s \models \varphi$; for *each* transition, compute t and show $t \models \varphi$ (split on the guard: taken vs. not taken).

Worked (state `int x:=0`, transition "`if x<m then x:=x+1`", $\varphi : \ 0 \leq x \leq m$): base $x = 0$ ok. Step: if $x < m$ then $x{+}1 \leq m$ and $x{+}1 > 0$; otherwise x unchanged. Either way $0 \leq x \leq m$, so φ is an inductive invariant.

Key Idea

Trap: a true property is often *not* inductive on its own. To fix it, **strengthen** it so it captures the relationship among *all* state variables (including the mode), as in φ_3 above.

Exam triage: *acyclic await?* draw awaited-input→output edges + wiring edges, check for cycle. *block diagram?* rename first, match names for internal arrows, unmatched = external, annotate types. *reachable set?* BFS from init. *invariant?* check all reachable. *inductive invariant?* disprove by any $s \models \varphi$ with bad successor; prove by base + step. *not inductive?* strengthen with mode constraints.

Chapter 4

Asynchronous Models

4.1 Core assumption

Definition

An asynchronous system places no fixed upper bound on message delay, process speed, or clock drift. Correctness must not depend on timing guarantees.

4.2 Typical properties

- Messages may be delayed arbitrarily long.
- A process may run much slower than its peers.
- Timeouts are only heuristics, not proof of failure.

4.3 Why it matters

Key Idea

Asynchrony is the most conservative model, so protocols that work here are usually robust under timing uncertainty.

4.4 Exam prompts

- Why can an asynchronous algorithm not rely on bounded delays?
- What is the practical impact of using timeouts in this model?
- Give an example of a problem that becomes harder in a fully asynchronous setting.

Chapter 5

Asynchronous Model

Asynchronous Model

Exam cheat-sheet: *process anatomy, parallel composition, fairness, Peterson, reachability*

5.1 What you are given (anatomy of a process)

An **asynchronous process** P has: input channels I, output channels O, state variables S (with `Init`), and a set of **tasks**. Every task is `guard` $\rightarrow$ `update`; a task fires only when its guard holds (it is *enabled*). **Exactly one task fires per step.** Three task kinds:

- **Input task** (on channel x): reads $S \cup \{x\}$, writes S. Action $s \xrightarrow{x?v} t$.
- **Internal task**: reads S, writes S. Action $s \xrightarrow{\varepsilon} t$ (silent).
- **Output task** (on channel y): reads S, writes $S \cup \{y\}$. Action $s \xrightarrow{y!v} t$.

A guard with no condition is always enabled. If several tasks are enabled, any one may be chosen (*interleaving*).

5.2 Parallel composition (the main exam question)

Format is always: two components, a composition expression like $A[\,out_A \mapsto temp\,] \mid B[\,in_B \mapsto temp\,] \setminus temp$, then parts (a)–(d). Read the expression like this:

- $[\,old \mapsto new\,]$ = **rename** that channel (relabelling), so the two components share a name.
- $\mid$ = compose; $\setminus temp$ = **hide** channel `temp` (it becomes internal, not visible outside).

(a) Connect the block diagram. External input $\rightarrow$ first component. Wherever an *output channel of one* was renamed to the *input channel of the other* (same new name, e.g. `temp`), draw an arrow between them. The remaining external output leaves the outer block.

(b) Label edges with the channel *type* and *name* (e.g. `int temp`). The hidden channel keeps its name but lives inside the outer box.

(c) List the tasks of the composition — this is the scored core. Rules:

- **Input tasks** = input tasks acting on channels still external (not hidden). Copy them unchanged.
- **Output tasks** = output tasks on channels still external. Copy them unchanged.
- **Internal tasks** = *each matched pair* on the hidden channel becomes *one* internal task: if A has output task `G1` $\rightarrow$ `U1` producing `temp`, and B has input task `G2` $\rightarrow$ `U2` consuming `temp`, the composition has

  $$(\texttt{G1 \& G2}) \rightarrow \texttt{U1; U2}$$

  Run U1 then U2; the `temp` value flows from the first into the second. Then **simplify** (substitute `temp` away — it is a local, not part of the state).
- Plus any *pre-existing* internal tasks of either component, unchanged.

(d) Give an execution for a given input (and sometimes output) sequence.

- State = tuple of the components' state variables, e.g. (x, y). `temp` is **not** in the state.
- One task per step. Label transitions $\xrightarrow{in?v}$ (input), $\xrightarrow{\varepsilon}$ (internal), $\xrightarrow{out!v}$ (output).
- Strategy: feed an input, fire the internal task to push the value through, then output when the target value is ready. Match the required output sequence exactly.

5.3 Fully worked template (Plus/Minus)

Given: `Plus`: $x{:=}0$; $P_1 : x{:=}x{+}inp$ (input), $P_2 : outp{:=}x$ (output). `Minus`: $y{:=}0$; $M_1 : y{:=}y{-}in_M$ (input), $M_2 : out_y{:=}y$ (output). Composition $\texttt{Plus}[outp \mapsto temp] \mid \texttt{Minus}[in_M \mapsto temp] \setminus temp$.

(c) Tasks of the composition:

- **input:** $x := x + inp$ $(= P_1$, still external$)$
- **internal:** $temp := x;\ y := y - temp \ \Rightarrow$ simplified $y := y - x$ (matched $P_2; M_1$ on `temp`)
- **output:** $out_y := y$ $(= M_2$, still external$)$

(d) Execution for input -1, 2, -1 and output 1, 0 (state (x, y)):

$$(0,0) \xrightarrow{inp?-1} (-1,0) \xrightarrow{\varepsilon} (-1,1) \xrightarrow{out_y!1} (-1,1) \xrightarrow{inp?2} (1,1) \xrightarrow{\varepsilon} (1,0) \xrightarrow{out_y!0} (1,0) \xrightarrow{inp?-1} (0,0)$$

(internal step computes $y := y - x$; output reads current y).

5.4 Fairness questions (short-answer)

For each *output* and *internal* task you may assume: **none**, **weak**, or **strong** fairness. Definitions to quote:

- **Weak fairness** to task A: if A is *almost always enabled* (continuously, with finitely many exceptions), then A is taken infinitely often.
- **Strong fairness** to task A: if A is *infinitely often enabled*, then A is taken infinitely often.

Key Idea

Decision rule: use **strong** fairness when the task keeps switching enabled↔disabled (its guard depends on a changing variable); **weak** fairness suffices when, once enabled, it stays enabled.

Quotable facts:

- Fairness restricts *infinite executions* only — it does **not** change the set of reachable states or any safety property.
- "Eventually"/liveness requirements may need fairness; safety ("never bad") never does.
- Merge process (two queues → one output): weak fairness on both output tasks suffices. Unreliable FIFO: strong fairness on the deliver task, none on loss/duplicate.

5.5 Mutual exclusion / Peterson facts (short-answer)

Two requirements to name: **Mutual exclusion** (never both in `Crit` — a *safety* property, provable by an invariant) and **deadlock freedom** (if someone wants in, someone gets in). Peterson uses `flag1, flag2, turn`: a process raises its flag, sets `turn` to the *other* ID, and waits until the other's flag is down *or* `turn` favours it.

Key Idea

To make "whoever wants in eventually gets in" hold, assume **weak fairness** on the entry tasks.

5.6 Reachability / `Post` (verifying a property)

To check whether property φ is an invariant, check that $\neg\varphi$ is *not reachable*. Compute reachable states by BFS: $reach_0 = \texttt{Init}$, $reach_{i+1} = reach_i \cup \texttt{Post}(reach_i)$, stop when no new states appear or $\neg\varphi$ is hit. **Symbolic** $\texttt{Post}(A, Trans)$ in 3 steps:

1. **Conjoin** region A with the transition formula `Trans` (over S and S').
2. **Existentially quantify** the old variables S (project them out).
3. **Rename** primed S' back to S.

Example: $A: \ 0 \le x \le 10$, $Trans: \ x' = 2x + 1$ gives $\texttt{Post}(A) = \{1 \le x \le 21\}$.

Exam triage: *composition tasks?* input/output = unchanged external tasks; internal = matched pair on hidden channel, simplified. *execution?* feed input, fire internal ε, emit output. *fairness?* strong if guard toggles, weak if it stays enabled. *invariant?* prove $\neg\varphi$ unreachable via `Post` BFS. *Peterson?* mutual exclusion (safety) + deadlock freedom; weak fairness for liveness.

Chapter 6

Safety Requirements

Safety Requirements

Exam cheat-sheet: *transition systems, invariants, inductive invariants, symbolic Post*

6.1 Core definitions to quote verbatim

A **transition system** $T = (S, \texttt{Init}, \texttt{Trans})$: state variables S, initial-state description `Init`, and transition description `Trans` (code updating S). Semantically: states Q_S, initial states $[\![\texttt{Init}]\!] \subseteq Q_S$, transitions $[\![\texttt{Trans}]\!] \subseteq Q_S \times Q_S$. Every synchronous/asynchronous component has an underlying transition system.

- **Reachable state:** a state s for which there is a *finite* execution from an initial state ending in s.
- **Invariant:** property φ such that *every reachable state* satisfies φ.
- **Inductive invariant:** property φ such that (1) every *initial* state satisfies φ, *and* (2) for every state s satisfying φ and every transition (s, t), the state t also satisfies φ (closed under transitions).

6.2 Three logical facts that answer most sub-questions

Key Idea

- Inductive invariant $\Rightarrow$ invariant. **Converse fails:** an invariant need not be inductive.
- The **set of reachable states** is itself an inductive invariant — in fact the *strongest* one. A formula that exactly describes the reachable set is always an inductive invariant.
- If φ is *not even an invariant*, it cannot be an inductive invariant (shortcut for part (c)).

6.3 The main exam question: Reachable States (a)–(e)

You are given an extended state machine (or synchronous component) and a formula φ_1. **Key hint: the mode is an implicit state variable**, so a state is the tuple $(\text{mode}, x, \dots)$.

(a) Provide the reachable part of the transition system. Start from the initial state (mode_0, x_0); repeatedly apply every enabled mode-switch/update; collect all states until no new ones appear. Draw them with arrows. (If a loop pumps a counter forever, the answer is *infinitely many* reachable states — still describe the pattern.)

(b) Is φ_1 an invariant? Check φ_1 on *every reachable state* from (a).

- All satisfy it $\Rightarrow$ **Yes**; justify: "all reachable states satisfy φ_1."
- One reachable state violates it $\Rightarrow$ **No**; name that state as the witness.

(c) Is φ_1 an inductive invariant? (Even if it *is* an invariant!)

- Look for *any* state s (reachable *or not*) that satisfies φ_1 but has a transition to a t that violates φ_1. If found $\Rightarrow$ **No**, give (s, t) as witness.
- If φ_1 failed (b), immediately answer **No** ("φ_1 is not even an invariant").
- If initial states satisfy φ_1 *and* no such bad (s, t) exists $\Rightarrow$ **Yes**.

(d) Give another invariant φ_2. Pick something weaker/looser that still holds on all reachable states but differs from φ_1 (e.g. widen a bound). Need not be inductive.

(e) Give another inductive invariant φ_3. Safe universal recipe: write the formula that *exactly describes the reachable set* (mode-by-mode). It is always inductive. Form:

$$(\text{mode} = A \wedge (\text{x-constraint for A})) \ \vee \ (\text{mode} = B \wedge (\text{x-constraint for B})) \ \vee \cdots$$

6.4 Fully worked template

ESM: modes A, B; `int x:=0`; edge $A \xrightarrow{x:=x+1} B$ and $B \xrightarrow{x:=x-1} A$. Let $\varphi_1 : x = 0 \vee x = 1$. State order (mode, x).

- **(a)** Reachable part: $(A,0) \to (B,1) \to (A,0) \to \cdots$, i.e. only $\{(A,0), (B,1)\}$.
- **(b) Yes** — both reachable states satisfy φ_1 ($x = 0$ and $x = 1$).
- **(c) No** — the state $(A,1)$ satisfies φ_1, but its successor $(B,2)$ does not. (Note $(A,1)$ is unreachable, but inductiveness must hold for *all* states satisfying φ_1.)
- **(d)** e.g. $\varphi_2 : x \geq 0$.
- **(e)** $\varphi_3 : (\text{mode} = A \wedge x = 0) \vee (\text{mode} = B \wedge x = 1)$ — exactly the reachable set.

Variant: $\varphi_1 : x < 5$, reachable set $\{(v, 42) \mid v \leq 5\}$. (a) infinitely many. (b) **No**, $(5, 42)$ is reachable but $5 \not< 5$. (c) **No**, not even an invariant. (d) $x \leq 6$. (e) $x \leq 5 \wedge \text{mode} = 42$.

6.5 Symbolic representation (transition formula)

Represent states by a formula over S, transitions by a formula φ_T over $S \cup S'$ (**primed = value after the step**).

- Assignment $x := e$ becomes $x' = e$. **Every unchanged variable must be stated explicitly**, e.g. $y' = y$.
- `if C then U1 else U2` becomes $(C \wedge \varphi_{U1}) \vee (\neg C \wedge \varphi_{U2})$.
- Guarded step `G` $\rightarrow$ `U` that may also idle: $(G \wedge \varphi_U) \vee (\neg G \wedge$ all vars unchanged$)$.

6.6 Checking an invariant by reachability (`Post` / BFS)

φ is an invariant $\iff$ the error set $\neg\varphi$ is **not reachable**. Compute reachable states by BFS:

$$reach_0 = [\texttt{Init}], \qquad reach_{i+1} = reach_i \cup \texttt{Post}(reach_i).$$

Stop when (i) $\neg\varphi$ is hit (invariant **fails**, counterexample found) or (ii) no new states appear (invariant **holds**).

Key Idea

Symbolic $\texttt{Post}(A, \texttt{Trans})$ in 3 steps:

1. **Conjoin** region A with `Trans` (intersect: all transitions starting inside A).
2. **Existentially quantify** the old vars S (project them out, leaving a formula in S').
3. **Rename** $S' \rightarrow S$.

Worked: $A: \ 0 \le x \le 10$, `Trans`: $x' = 2x + 1$. Conjoin $\rightarrow (0 \le x \le 10) \wedge (x' = 2x + 1)$; eliminate x via $x = (x' - 1)/2 \rightarrow 1 \le x' \le 21$; rename $\rightarrow \boxed{1 \le x \le 21}$.

Exam triage: *reachable states?* BFS from init, apply all transitions. *invariant?* check every reachable state. *inductive invariant?* also check non-reachable states satisfying φ — find bad (s, t) pair. *inductive invariant for (e)?* describe reachable set mode-by-mode. *Post?* conjoin, existentially quantify S, rename $S' \rightarrow S$.

Chapter 7

Safety Requirements — Part 2

Safety Requirements — Part 2

Exam cheat-sheet: *safety monitors, symbolic verification, Post/BFS, data structures*

7.1 Safety monitor: setup to quote

A **safety monitor** M watches a system's behavior and moves to an **error mode** when something forbidden happens. As an extended state machine:

- The **inputs of** M = the input/output variables of the system being monitored (it reads what the system does).
- M has **no output back into the system** — it observes, never influences.
- A subset F of modes is declared **accepting/error**. "Bad behavior" = an execution that drives M into F.

Key Idea

Verification: the property holds $\iff$ "M.mode $\notin F$" is an **invariant** of the composition $C \parallel M$. A safety property is thereby reduced to an invariant check.

7.2 The main exam question: Safety Monitor (a)–(b)

You are given an output alphabet and a forbidden-pattern property in words (e.g. "a 1 is never immediately followed by a 0"), then:

(a) Design the monitor as an extended state machine. Recipe:

1. Decide what you must *remember* to detect the bad pattern — usually "how much of the forbidden prefix have I seen?" Make one mode per such situation.
2. **Start mode** = nothing dangerous seen yet.
3. Add a mode for each partial match of the bad pattern (e.g. "the last output was 1").

4. The transition that *completes* the forbidden pattern goes to the **error mode** e (this is F).
5. Make e a **sink**: all symbols loop back to e (once bad, stays bad).
6. Every mode needs an outgoing edge for *every* symbol of the alphabet (the monitor is total/deterministic).

(b) Does a given component satisfy the property?

- **Yes:** argue that no execution of the component can produce the forbidden output sequence (e.g. inspect the component's reachable outputs or structure).
- **No:** give the **prefix of a counterexample execution** — a concrete input/output sequence of the component that produces the forbidden pattern. Provide states and outputs as in execution-trace questions.

7.3 Fully worked template

Output alphabet $\{0, 1, 2\}$. Property: "output 1 is never immediately followed by output 0."

(a) Monitor with modes s (start), q_1 (last output was 1), e (error, $F = \{e\}$):

- $s \xrightarrow{0,2} s$ (no pending 1)
- $s \xrightarrow{1} q_1$ (just saw a 1)
- $q_1 \xrightarrow{1} q_1$ (still ends in 1)
- $q_1 \xrightarrow{2} s$ (1 not followed by 0; safe)
- $q_1 \xrightarrow{0} e$ (1 **then** 0 **= violation**)
- $e \xrightarrow{0,1,2} e$ (sink)

Verify: check that "mode $\neq e$" is an invariant of $C \parallel M$.

(b) A length-3 counterexample prefix that emits $\ldots 1$ then 0 drives the monitor to e; provide the state sequence with outputs.

7.4 Symbolic verification machinery (short-answer)

Region = a set of states represented as a formula over the variables. A transition system is represented symbolically by:

- φ_I over S for initial states; φ_T over $S \cup S'$ for transitions (primed = next value).
- Build φ_T from reaction code: assignments become $x' = e$, unchanged vars add $y' = y$, then **existentially quantify out** local/input/output variables.

Operations on regions (for formulas):

- $\texttt{Conj}(A, B) = A \wedge B$; $\texttt{Disj}(A, B) = A \vee B$; $\texttt{Diff}(A, B) = A \wedge \neg B$.
- $\texttt{IsEmpty}(A)$ = satisfiability (SAT) test; $\texttt{Rename}(A, X, Y)$ = textual substitution.
- $\texttt{Exists}(\varphi, x) = \varphi[x \mapsto 0] \vee \varphi[x \mapsto 1]$ for a Boolean x (quantifier elimination).

7.5 Checking an invariant symbolically (`Post` + BFS)

φ is an invariant $\iff$ the error set $\neg\varphi$ is **unreachable**.

Key Idea

$\texttt{Post}(A, \texttt{Trans}) = \texttt{Rename}(\texttt{Exists}(\texttt{Conj}(A, \texttt{Trans}), S), S', S)$ — **(1)** conjoin A with `Trans`, **(2)** existentially quantify the old vars S, **(3)** rename $S' \to S$.

Symbolic BFS: `Reach := Init; New := Init;`
while `New` not empty: if $\texttt{Conj}(\texttt{New}, \varphi_{err})$ nonempty $\Rightarrow$ report *reachable* (property fails);
else $\texttt{New} := \texttt{Diff}(\texttt{Post}(\texttt{New}), \texttt{Reach})$, $\texttt{Reach} := \texttt{Disj}(\texttt{Reach}, \texttt{New})$.
If `New` empties with no error hit $\Rightarrow$ property holds.

Worked `Post`: $A: \ 0 \le x \le 10$, `Trans`: $x' = 2x + 1$ $\Rightarrow$ conjoin, eliminate x, rename $\Rightarrow \boxed{1 \le x \le 21}$.

7.6 Data structures (one-liners)

- Naive formula representation grows without bound and needs SAT (NP-complete) for `IsEmpty`.
- **Boolean** variables $\to$ **(RO)BDDs** (Reduced Ordered Binary Decision Diagrams).
- **Clocks** (real, timed automata) $\to$ **DBMs** (Difference Bound Matrices); a *zone* = conjunction of constraints $x \sim n$ and $x - y \sim n$.
- General real-valued variables $\to$ polyhedra.

Exam triage: *design monitor?* one mode per partial bad-pattern match; error sink; every symbol has an edge from every mode. *verify?* check "mode $\neq e$" is invariant of $C \parallel M$. *counterexample?* concrete trace reaching error mode. *Post?* conjoin, $\exists$-quantify S, rename $S' \to S$. *data structure?* Booleans $\to$ BDD; clocks $\to$ DBM; reals $\to$ polyhedra.

Chapter 8

Timed Models

8.1 Core assumption

Definition

A timed model adds explicit timing information to distributed behavior, often through deadlines, timestamps, or bounded-delay constraints that are weaker than full synchrony but stronger than pure asynchrony.

8.2 Typical properties

- Time is part of the specification.
- Correctness may depend on meeting deadlines.
- Timing constraints can be partial rather than global.

8.3 Why it matters

Key Idea

Timed models are useful when real systems need temporal guarantees but cannot assume every operation is perfectly synchronized.

8.4 Exam prompts

- Distinguish timed models from synchronous models.
- Explain why deadlines are central in this model.
- Describe one application where timing is a correctness concern.

Chapter 9

The Timed Model

The Timed Model

Exam cheat-sheet: *timed processes, timed transitions, modelling delays, composition, Fischer*

9.1 What a timed process is

Key Idea

Timed process $TP = (P, CI)$ = an **asynchronous process** P (some state vars are `clock`) + a **clock invariant** CI (a Boolean condition over the clocks).

Inputs, outputs, states, initial states, and actions (internal/input/output) are defined exactly as in the asynchronous model. The new ingredient is **clocks**: real-valued, ≥ 0, all tick at the same rate, can be tested and reset (`x:=0`) on mode-switches. Called the "*semi-synchronous*" model (mix of async actions + synchronous time).

9.2 The two kinds of transition

A **state** is (mode, clock values), e.g. $(\text{off}, x{=}0.5)$.

- **Timed (delay)** $s \xrightarrow{\delta} s{+}\delta$: time $\delta > 0$ passes; *every* clock grows by δ, non-clock variables unchanged.
- **Discrete (action)** $s \to s'$: take an edge (input `?`, output `!`, or internal). Guard must hold; apply resets. Takes **zero** time.

Key Idea

When is a delay δ legal? The clock invariant CI must stay true for the *whole* interval $[0, \delta]$. For a convex CI (the usual case) it is enough to check the **endpoints** s and $s{+}\delta$.

Example (invariant $y \leq 2$): from $(\text{Full}, y{=}0.8)$ a delay of $0.7 \to (\text{Full}, y{=}1.5)$ is **allowed** ($1.5 \leq 2$); a delay of $1.4 \to (y{=}2.2)$ is **disallowed** ($2.2 > 2$).

9.3 Tracing a run (core exam skill)

A run alternates timed and discrete transitions. **Recipe:**

1. Start at the initial state (mode + all clocks at their init values).
2. **Timed step**: add δ to every clock; check the current mode's invariant holds throughout.
3. **Discrete step**: pick an enabled edge (guard true); update mode, apply clock resets, leave other clocks unchanged.
4. Repeat; write each state in the chain.

Worked example – light switch (clock x; `off->dim` resets $x{:=}0$; `dim->bright` guard $x \leq 1$):

$$(\text{off}, 0) \xrightarrow{0.5} (\text{off}, 0.5) \xrightarrow{press?} (\text{dim}, 0) \xrightarrow{0.8} (\text{dim}, 0.8) \xrightarrow{press?} (\text{bright}, 0.8)$$

The last edge needs $x \leq 1$: $0.8 \leq 1$ ✓, and `bright` keeps x (no reset).

Two clocks grow *together*: $(\text{Wait2}, x{=}0.8, y{=}0) \xrightarrow{0.3} (1.1, 0.3) \xrightarrow{0.72} (1.82, 1.02)$ – the difference $x - y$ never changes during a delay.

```
def delay(clocks, d, invariant): # clocks = {'x':0.8,'y':0.0}
    new = {c: v + d for c, v in clocks.items()}
    assert invariant(new), "invariant violated by delay"
    return new

def take_edge(clocks, guard, resets): # resets = ['x']
    assert guard(clocks), "guard not satisfied"
    return {c: (0.0 if c in resets else v) for c, v in clocks.items()}

# light switch: off -(0.5)-> press? (reset x) -(0.8)-> press? (need x<=1)
s = {'x': 0.0}
s = delay(s, 0.5, lambda c: True) # (off, 0.5)
s = take_edge(s, lambda c: True, ['x']) # press? -> (dim, 0)
s = delay(s, 0.8, lambda c: True) # (dim, 0.8)
s = take_edge(s, lambda c: c['x'] <= 1, []) # press? -> (bright, 0.8)
print(s) # {'x': 0.8}
```

9.4 Modelling a timing spec (guard + invariant pattern)

To express "after event, output must happen in the interval $[L, U]$":

Key Idea

Guard $x \geq L$ on the exit edge = *earliest* it may fire (lower bound). **Invariant** $x \leq U$ on the waiting location = *latest* it may stay (upper bound, **forces** it out).

A guard alone only *enables* an edge – it never *forces* action; to guarantee the upper bound you **must** add the location invariant. On entry, reset the clock (`x:=0`).

Example spec: on `i?`, emit `u!` in $[2, 4]$ and `v!` in $[3, 5]$. Reset $x{:=}0$ on `i?`; give the wait-location invariant $x \leq 4$ and the `u!` edge guard $x \geq 2$ (similarly $x \leq 5$ / $x \geq 3$ for `v!`).

9.5 Parallel composition of timed processes

Key Idea

$$TP_1 \,|\, TP_2 = (P_1 \,|\, P_2,\ \ CI_1 \wedge CI_2) \quad \text{(defined iff outputs are disjoint, as in async).}$$

Consequence (key): a timed step of duration d is allowed **only when both** components are willing to wait d – i.e. the *tighter* invariant wins ("earliest deadline wins"). The discrete actions interleave / synchronize exactly as in the asynchronous model.

Two timed buffers: composite invariant $= (y_1 \leq UB_1) \wedge (y_2 \leq UB_2)$, so the buffer with the smaller bound forces the next move.

9.6 From timed process to timed automaton

Key Idea

A timed process is a **timed automaton** when, for every clock x: the only assignment is `x:=0`, and clocks appear only as $x \sim k$ ($\sim \in \{<, \leq, =, \geq, >\}$, k constant) in guards/invariants.

This lets you express constant lower/upper delay bounds, and is **closed under composition**. A **finite-state** timed automaton additionally has all non-clock variables of finite type – the state space is still infinite (clocks are real) but verification is exactly solvable.

9.7 Fischer's protocol (timing-based mutual exclusion)

Shared variable `Turn` (0 = free) + a per-process clock x with two timing constants. Path:

`Idle` → `Test` → `Set` → `Delay` → `Check` → `Crit`.

- `Test`: if `Turn=0`, reset x:=0 and go to `Set`; else retry.
- `Set` (invariant $x \leq \Delta_1$): write `Turn:=myID` within Δ_1 (max write time), then x:=0.
- `Check` (guard $x \geq \Delta_2$): wait at least Δ_2, then re-read: `Turn=myID`? → `Crit`; else retry.

Key Idea

Correct **iff** $\Delta_2 > \Delta_1$. Then: mutual exclusion + deadlock-freedom hold. (Not starvation-free.) Works for any number of processes.

Verify in UPPAAL: `A[] not (P1.Crit and P2.Crit)`.

Exam triage: *trace a run?* alternate delay (all clocks $+\delta$, check invariant) and edge (guard true, apply resets). *delay legal?* endpoints satisfy CI. *model delay* $[L,U]$*?* edge guard $x \geq L$ + location invariant $x \leq U$ + reset on entry. *compose?* $CI_1 \wedge CI_2$, tighter invariant wins. *is it a timed automaton?* only `x:=0` and `x`$\sim$`k`. *Fischer?* needs $\Delta_2 > \Delta_1$, check `A[] not (P1.Crit and P2.Crit)`.

Chapter 10

Timed Automata & UPPAAL

Timed Automata & UPPAAL

Exam cheat-sheet: *clocks, guards/invariants, reachability & zones, UPPAAL queries*

10.1 Anatomy of a timed automaton

A finite graph + real-valued **clocks** that all tick at the same rate.

- **Locations** (circles; the book calls them "modes").
- **Clocks** $x, y, \ldots$ – start at 0, increase with time, can be reset.
- **Guard** on an edge: condition that must hold to *take* the edge, e.g. `x>=3`.
- **Reset** on an edge: `x:=0` (only allowed assignment to a clock).
- **Invariant** on a location: condition that must stay true to *remain* there, e.g. `y<=2`.
- **Sync label**: `c!` / `c?` for channel communication (see §10.6).

Key Idea

Timed-automaton restriction: clock terms only appear as $x \sim k$ ($\sim \in \{<, \leq, =, \geq, >\}$, k a constant), and the only clock assignment is `x:=0`. (Other variables must have finite types.)

10.2 Guards vs. invariants (don't mix these up)

Key Idea

Guard = *lower-bound enabler* on an edge ($x \geq k$: "not before"). **Invariant** = *upper-bound deadline* on a location ($x \leq k$: "must leave by").
Together they form a **time window** $[\, k_{\text{guard}},\, k_{\text{inv}} \,]$ in which the edge can fire.

10.3 State & the two kinds of transition

A **state** is a pair (ℓ, v): location ℓ + clock valuation v (e.g. $x{=}2, y{=}1$).

- **Delay** $(\ell, v) \xrightarrow{\delta} (\ell, v{+}\delta)$: time δ passes, *every* clock grows by δ. Allowed only if the location invariant stays true the whole time.
- **Action** $(\ell, v) \to (\ell', v')$: take an edge – its guard must hold now; apply resets to get v'. Takes *zero* time.

Notation: $\xrightarrow{2}$ = delay 2; plain $\to$ = take an edge.

10.4 The recurring exam exercise (Exploration of Timed Automata)

Given an automaton with clocks x, y and goal locations, you must answer:

(a) Is location L reachable? Try to find *any* run (delays + edges) from $(\text{init}, x{=}0, y{=}0)$ to L. If a guard can never be satisfied on every path, it is unreachable.

(b) Give a timed transition sequence. Write the alternating delay/action run, e.g.

$$(A,0,0) \xrightarrow{1} (A,1,1) \to (B,1,0) \xrightarrow{3} (B,4,3) \to (C,4,3) \to (E,4,3).$$

(c) Fastest time to reach L + witness.

1. At every location delay the *minimum* needed to satisfy the next guard's lower bound – no extra waiting.
2. Sum all the delays along that run = the answer; the run itself is the witness.

(In UPPAAL: Options → Diagnostic trace → Fastest, then verify `E<> L`*.)*

(d) Reachable zones "upon entry" and "after delay" (as difference constraints):

- **Entry zone** = valuations the moment you arrive (right after the incoming edge's resets).
- **After delay** = apply *delay*: let time pass, bounded by the location invariant. Geometrically the entry region slides *diagonally up-right* (both clocks $+\delta$) until the invariant stops it.

Example: A entry $x{=}0 \wedge y{=}0$, after delay $x{=}y \wedge 0 \le x \le 2$; C entry $x \ge 4 \wedge y \le 4 \wedge 1 \le x{-}y \le 2$, after delay same but $y \le 5$.

(e) Weaken a guard to make L reachable. Loosen the blocking bound.

> **Key Idea**
>
> "Weaker" = lets *more* values through: `x<=7` is weaker than `x<=5`; `x>=2` is weaker than `x>=4`.

Find the guard that blocks every path to L and relax it just enough (e.g. change `y<=2` to `y<=4`).

10.5 Zones & difference constraints

A **zone** = a conjunction of constraints of the form $x \sim k$ and $x - y \sim k$ (a "difference constraint"). It compactly represents infinitely many clock valuations. The clock *difference* $x - y$ is fixed by delay (both grow together) and only changes on a reset. UPPAAL stores zones as **DBMs** (Difference Bound Matrices).

- **Delay**: drop upper bounds on individual clocks (keep differences) up to the invariant.
- **Guard**: intersect the zone with the guard.
- **Reset** `y:=0`: set y to 0 (project onto the x-axis).

10.6 Networks, channels & special locations

Several automata run in parallel; they synchronize on channels.

- **Binary channel** `c`: one `c!` (send) fires together with one `c?` (receive), at the same instant.
- **Broadcast channel**: one `c!` fires with *all* ready `c?` receivers at once (one-to-many). A sender with no receivers still fires.
- **Urgent location**: time may not pass here (leave immediately, 0 delay).
- **Committed location**: like urgent, but the *next* transition must come from a committed location – used for atomic multi-step updates.

10.7 UPPAAL query language

P, Q are state conditions like `Train.Crossing`, `x>=5`, `buffer<=0`.

Meaning	Query	Reading
Possibly (reachable)	`E<> P`	some run reaches a state with P
Invariant / safety	`A[] P`	P holds in *all* states of *all* runs
Possibly-invariant	`E[] P`	some run keeps P true forever
Eventually	`A<> P`	every run eventually reaches P
Leads-to	`P -> Q`	whenever P, then later Q
No deadlock	`A[] not deadlock`	system never gets stuck
Max value	`sup{cond}: expr`	largest `expr` over states with `cond`

Common patterns:

```
E<> Task.Done and time<=30 // can finish within 30?
A[] not (T0.Error or T1.Error) // no task ever misses a deadline
A[] player.buffer <= 4000 || time >= SECOND // bound holds in 1st second
sup{time <= SECOND}: player.buffer // biggest buffer in 1st second
```

Minimum completion time: binary-search the bound K in `E<> all.Done and time<=K`, *or* ask for the fastest diagnostic trace of `E<> all.Done` and read `time` in the simulator.

10.8 Fischer's mutual exclusion (timed protocol)

Each process uses a shared `id` variable + a clock with the same bound k on *wait* (guard) and *set* (invariant). Timing guarantees only one process enters the critical section.

Key Idea

Verify mutual exclusion: `A[] not (P1.cs and P2.cs)` (no two processes in the critical section together).

Exam triage: *reachable?* find any run / `E<> L`. *fastest?* delay the minimum each step, sum delays. *zone after delay?* slide entry region diagonally up to the invariant. *weaken guard?* loosen the blocking bound (`<=` bigger / `>=` smaller). *safety?* `A[] not bad`. *liveness?* `A<> good` or `P -> Q`. *deadlock?* `A[] not deadlock`. *max value?* `sup{...}`.

Chapter 11

Continuous Models

11.1 Core assumption

Definition

A continuous model represents system evolution over a continuum of time, so state changes are described using trajectories, rates, and dynamics rather than only discrete steps.

11.2 Typical properties

- State changes smoothly over time.
- Derivatives, rates, and invariants are often central.
- The model is common in control and physical systems.

11.3 Why it matters

Key Idea

Continuous models are useful when the system interacts with physical processes, where discrete event abstraction is too coarse.

11.4 Exam prompts

- Explain the difference between continuous and discrete-time reasoning.
- Give an example of a system naturally modeled continuously.
- Describe what it means for a property to hold over an interval of time.

Chapter 12

Dynamical Systems – Part 1

Dynamical Systems – Part 1

Exam cheat-sheet: *continuous-time model, Euler / Runge-Kutta, equilibria & stability*

A continuous-time component is the "physics box". To **specify** one, list:

- **Inputs** I: real-valued, type `real` or interval `real[L,U]`.
- **Outputs** O and **state** S: real-valued.
- **Init**: the set of allowed start states $[\![Init]\!]$.
- For each output y: an algebraic expression h_y over $I \cup S$ $(y(t) = h_y)$.
- For each state x: a derivative expression f_x over $I \cup S$ $(\frac{dx}{dt} = f_x)$.

An **execution** given an input signal $I(t)$ is a state signal $S(t)$ + output signal $O(t)$ with: (1) $S(0) \in [\![Init]\!]$; (2) $y(t) = h_y(I(t), S(t))$; (3) $\frac{d}{dt}x(t) = f_x(I(t), S(t))$.

Exam tip: state variables get a **derivative** $(dx/dt = \dots)$; outputs get a plain **equation** $(y = \dots)$. Example car: state v with $dv/dt = (F - kv)/m$, input force F.

12.1 Does a solution exist / is it unique?

Initial value problem (IVP): $\dfrac{dx}{dt} = f(x), \quad x(0) = x_0.$

Key Idea

Existence (at least one solution): f is **continuous**.
Uniqueness (exactly one solution) – *Picard–Lindel"of*: f is **Lipschitz-continuous**.

Lipschitz test: f is Lipschitz if there is a constant K with $|f(u) - f(v)| \leq K\,|u - v|$ for all u, v (a finite bound on how fast f changes).

Quick classification (memorize these)

- Linear, e.g. $(F - kv)/m$ $\Rightarrow$ **Lipschitz** (always).
- x^2 (any polynomial) $\Rightarrow$ Lipschitz **only on a bounded domain**.
- $x^{1/3}$ $\Rightarrow$ **not** Lipschitz at 0 $\Rightarrow$ IVP $dx/dt = x^{1/3}, x(0) = 0$ has *multiple* solutions.
- $f(x) =$ `if` $x = 0$ `then` 1 `else` 0 $\Rightarrow$ **not** continuous $\Rightarrow$ *no* solution (jumps $\rightarrow$ hybrid systems).

12.2 Euler method (the main computation)

Numerically solve $\dfrac{dx}{dt} = f(x,t)$ from x_0 in steps of size h.

Key Idea

$$\tilde{x}_{i+1} = \tilde{x}_i + h \cdot f(\tilde{x}_i, t_i), \qquad t_{i+1} = t_i + h, \qquad \tilde{x}_0 = x_0.$$

Recipe (fill the table):

1. Write the starting row: $i = 0,\ t_0,\ \tilde{x}_0$.
2. Compute slope $f(\tilde{x}_i, t_i)$ at the *current* row.
3. New value = old value $+ h\times$ slope. Advance t by h.
4. Repeat until $i = N$.

Worked example ($\frac{dx}{dt} = t$, $x_0 = 0$, $t_0 = 0$, $h = 1$, $N = 5$; exact $x(t) = \frac{1}{2}t^2$):

i	t_i	$\tilde{x}_{i+1} = \tilde{x}_i + h \cdot f$	exact $x(t_i)$
0	0	$0 + 1 \cdot 0 = 0$	0
1	1	$0 + 1 \cdot 1 = 1$	0.5
2	2	$1 + 1 \cdot 2 = 3$	2
3	3	$3 + 1 \cdot 3 = 6$	4.5
4	4	$6 + 1 \cdot 4 = 10$	8
5	5	—	12.5

Here f depends only on t, so the slope is just t_i. If f depends on x (e.g. $dx/dt = -x$), plug in $\tilde{x}_i$ instead.

```
def euler(f, x0, t0, h, N):
    x, t = x0, t0
    for i in range(N + 1):
        print(i, round(t, 3), round(x, 4))
        x = x + h * f(x, t) # Euler update
        t = t + h
euler(lambda x, t: t, 0, 0, 1, 5) # dx/dt = t
# for dx/dt = -x use: lambda x, t: -x
```

12.3 Runge–Kutta & truncation error

Euler uses only the slope at the *start* of the interval; Runge–Kutta also looks ahead.

2nd-order Runge–Kutta (average of two slopes):

$$k_1 = f(\tilde{x}_i), \quad k_2 = f(\tilde{x}_i + h\,k_1), \quad \tilde{x}_{i+1} = \tilde{x}_i + h\,\tfrac{k_1+k_2}{2}.$$

4th-order Runge–Kutta (most used): $\tilde{x}_{i+1} = \tilde{x}_i + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$.

Key Idea

Global truncation error (how far $\tilde{x}_i$ is from the true $x(t_i)$):
Euler: $|\tilde{x}_i - x(t_i)| \le C\,h$ (error $\propto h$). RK4: $|\tilde{x}_i - x(t_i)| \le C\,h^4$ (error $\propto h^4$).
Smaller $h \Rightarrow$ smaller error. Since $h^4 < h$ for $0 < h < 1$, RK4 is more accurate than Euler.

12.4 Equilibria

Key Idea

A state x_e is an **equilibrium** of $\dfrac{dx}{dt} = f(x)$ iff $f(x_e) = 0$.

Recipe: set the derivative expression(s) to 0 and solve for the state. If you start exactly at x_e, the system stays there forever (constant signal). *Pendulum:* two equilibria – hanging down ($\varphi = 0$) and balanced straight up ($\varphi = -\pi$).

12.5 Stability (Lyapunov)

You perturb the system slightly away from x_e – what happens?

Key Idea

Stable	stays *near* x_e forever (small nudge → small wobble).
Asymptotically stable	stable *and* returns to x_e as $t \to \infty$ (stronger).
Unstable	drifts *away* from x_e.

Formal: x_e *stable* if $\forall \varepsilon > 0\, \exists \delta > 0$: $|s - x_e| < \delta \Rightarrow |x[s](t) - x_e| < \varepsilon\ \forall t \ge 0$. *Asymptotically stable* adds $\lim_{t\to\infty} x[s](t) = x_e$.
Pendulum intuition: hanging down = stable (frictionless) / asymptotically stable (with friction); balanced up = unstable.

One-line exam triage: "run Euler" → table with $\tilde{x}_{i+1} = \tilde{x}_i + hf$; "find equilibria" → solve $f(x) = 0$; "unique solution?" → check Lipschitz; "error vs h" → Euler $\propto h$, RK4 $\propto h^4$; "classify x_e" → stable / asymptotic / unstable.